FUZE

Design Report on Software Maintainability

Version 2.0

By: Jerral Chua Fung Yong, Kataruka Mohika, Jethani Samarth, Sinha Armaan, Jain Sparsh, Faye Chan and Lai Yi

Document Change Record

Revision	Description of Change	Approved by	Date
1.0	Initial draft covering FUZE design strategies, architecture and configuration management.	Kataruka Mohika	14/10/2026
2.0	Revised the design strategies and architecture descriptions to better reflect the application's implementation.	Kataruka Mohika	20/10/2026

Contents

1. Design Strategies 3
1.1. The Planning Phase Before Development 3
1.2. The Process of Developing 4
1.3. Correction by Nature 5
1.3.1. Corrective Maintainability 5
1.3.2. Preventive Maintainability 5
1.4. Enhancement by Nature 6
1.4.1. Adaptive Maintainability 6
1.4.2. Perfective Maintainability 6
1.5. Maintainability Practices 6
2. Architectural Design Patterns 8
3. Software Configuration Management Tools 12
3.1. GitLab 12
3.2. Trello 12
3.3. Google Drive and Controlled Documentation 12

1. Design Strategies

FUZE validates thecompatibilityandsourceabilityofelectronicboardsatblock level. Engineers submit an English design brief or an existing bill of materials (BOM). The web application retrieves real component data, applies ten deterministic constraint rules and supports model-assisted repairs, a live validation trace and saved results.

The design separates presentation, sourcing, workflow coordination and validation so that changes can be isolated and tested. This report explains the resulting architecture and the practices used to diagnose faults, manage changes and maintain the application.

1.1. The Planning Phase Before Development

Planning defines the scope, shared data contracts and team responsibilities before independent development. The Engineer uses the application, supported by external catalogue and model services. The sixteen use cases cover the capabilities below.

Functional scope of FUZE

Feature area	Capabilities
Accounts and projects	Register, sign in, restore a session and sign out. Create, browse, rename and delete owned projects, and browse their runs. Cancelled or failed changes must leave an accurate saved state.
Guided walkthrough	Introduce a new engineer to the application through a demonstration run.
Board design and inspection	Interpret a brief, plan component slots and rails, source parts and validate the board. Display the live or completed trace, component logic graph, conflict drawer and BOM.
Repair and recovery	Apply permitted repairs, escalate unresolved questions to the engineer and resume after an answer. Continue an abandoned or errored run from its saved checkpoint.
Saved boards and export	View a saved board without restarting its workflow and export the final post-repair BOM as CSV.
Existing BOM validation	Validate a supported CSV file or manufacturer part numbers entered one per line, independently of the board-design workflow.
Datasheet attachment	Upload a manufacturer PDF with a part number and package to supply a missing thermal value, retaining the extracted source line.
Memory and analytics	Browse remembered parts, associated boards and findings. Review rule outcomes, repair effectiveness, recurring conflicts and supply exposure from completed runs.

The board is represented through component slots, placed parts and power rails. Shared contracts define the brief, typed part specifications, rule verdicts, ordered

events and stored run state. Each layer must preserve the same units and the meaning of missing data. Establishing these contracts early limits integration errors and makes later changes easier to review.

FUZE operates at block level. Schematic capture, PCB layout and routing, circuit simulation and manufacturing-file generation are outside its scope. Maintenance should focus on the existing workflow; proposed extensions require an assessment of their effect on interfaces, testing, effort and schedule before the project scope changes.

1.2. The Process of Developing

Development proceeds incrementally from planning and system design through implementation, integration, testing and refinement. The core flow connects a brief to sourcing, validation, streamed progress and a final BOM. Account management, recovery, uploads, memory and analytics are integrated around this flow using the shared contracts.

Each maintenance task should identify the affected requirement or defect, the module responsible and any change to stored data or interfaces. The developer makes the change on a short-lived branch, updates the relevant documentation and adds regression coverage. Another team member reviews the result before merge; changes to the engine or interface contract also require the technical lead's review.

Verification follows the architecture. Rule tests check arithmetic, boundary conditions and missing inputs. Recorded supplier responses support repeatable adapter tests. Integration tests cover workflow state, persistence, event ordering and analytics, while browser tests exercise the user-facing use cases. Prompt-injection and authorisation tests protect the boundary between model suggestions, validation decisions and owned data.

The quality requirements include at least 80% backend coverage and 95% constraint-engine coverage. Repeated evaluation of identical stored inputs must produce identical verdicts. Performance targets are a validated BOM within 130 seconds with live data and 35 seconds with recorded data for up to twelve slots. Release acceptance requires retained test results for these criteria.

Maintenance ownership follows the team's delivery responsibilities. Sparsh leads technical design and engine review; Armaan owns backend integration; Lai Yi owns the frontend; Samarth leads verification; Faye leads validation; and Jerral handles release engineering. Mohika coordinates changes, risks and sponsor communication. A second reviewer and shared handover notes reduce dependence on any single developer.

1.3. Correction by Nature

Correction keeps FUZE working according to its requirements. Corrective maintenance addresses faults that have occurred, while preventive maintenance reduces the likelihood of future faults. Both depend on clear module boundaries and reproducible validation results.

1.3.1. Corrective Maintainability

Corrective maintenance restores expected behaviour when a feature produces an incorrect result. In FUZE, the stored board, validation trace and source information help locate the cause. For example, an incorrect current-budget result can be checked against the recorded component currents and the required 15% margin. A missing current value must remain unchecked rather than being treated as zero. If the error comes from a catalogue conversion, the correction belongs in normalisation instead of being hidden inside the rule.

The same approach applies to broken session handling, incorrect upload validation, inaccurate analytics counts and stalled event streams. The team should reproduce the fault, correct the responsible component and add a regression case. Stream recovery must also preserve event ordering and repair counters. A second team member should verify the correction before the defect is closed.

1.3.2. Preventive Maintainability

Preventive maintenance strengthens the application before weaknesses become failures. FUZE separates the deterministic constraint engine from supplier requests and model calls, allowing rules to be checked independently. Typed specifications, explicit missing values and stable event contracts reduce ambiguity between modules. Legal-action checks and repair limits also keep model suggestions within the permitted workflow.

Changes should be supported by code review and tests for normal, boundary and missing-data conditions. Recorded supplier responses allow repeatable checks without depending on live services. Integration tests should cover authorisation, uploads, analytics and stream recovery. Keeping dependency versions, setup instructions and design decisions with the code helps future maintainers reproduce failures and reduces reliance on one developer.

1.4. Enhancement by Nature

Enhancement keepsFUZEusefulasitsenvironmentanduserneedschange.
Adaptive maintenance responds to external changes, while perfective maintenance improves the performance and usability of existing functions.

1.4.1. Adaptive Maintainability

FUZE relies on JLCPCB, Mouser, manufacturer datasheets, a model service and hosted infrastructure. If a supplier changes its response format, the relevant adapter and normalisation mapping can be updated while preserving the engine's typed input contract. Field names, units and provenance must still be checked. Changes to normalisation instructions must also update the cache fingerprint so that an outdated interpretation is not reused.

Updates to React, FastAPI or LangGraph require regression checks around sessions, streaming and workflow recovery. Hosting or PostgreSQL changes require corresponding runbook updates and checks that existing records remain usable. The documented idempotent schema statements support repeated application startup; changes to them still require review of restart behaviour and reversal procedures.

1.4.2. Perfective Maintainability

Perfective maintenance improves how efficiently engineers complete and understand the existing workflow. Performance work can focus on concurrent sourcing, narrower supplier queries and reuse of normalised physical properties. Stock, lifecycle information and datasheet links must still be retrieved live for each run. Timing individual stages helps identify whether a slowdown comes from sourcing, normalisation or repair.

Interface improvements can clarify loading states, conflict explanations, BOM organisation and retry or resume actions. The walkthrough and upload feedback can help new users complete tasks with less assistance. Memory and analytics views can make previous findings and recurring conflicts easier to understand, while retaining the counts and unchecked information behind each result. These improvements should make the evidence easier to interpret without changing the engine's validation decisions.

1.5. Maintainability Practices

FUZE's maintenance practices keep the application understandable,changes traceable and releases reproducible. They protect the connection between engineering evidence, validation results and the interface while allowing individual components to evolve.

- **Readable Code**

The constraint engine defines ten rules as pure functions over typed board and part data. Inputs, units, calculations and outcomes must be explicit, with the deciding field and its source retained in each verdict. Rules must not depend on network services, language-model calls or the system clock. Missing required data must

produce an unchecked result identifying the missing field. This makes faults easier to locate and decisions easier to reproduce. Rule changes require independent arithmetic checks and regression cases covering passing, failing, boundary and missing-data conditions.

- **Version Control**

The campus GitLab repository is the controlled location for source code, schemas, tests, configuration and reviewable documentation. Changes should use short-lived branches and undergo review by another engineer; engine and interface changes also require the technical lead's review. GitLab issues and Trello connect changes to defects, owners and milestones. Each release must identify its tested commit, dependency versions and database schema state. Required checks must pass before changes are merged. Release tags and an up-to-date local clone support recovery, while credentials remain in protected environment settings.

- **Standardized Documentation**

Requirements, API and event contracts, rule formulas, repair limits and test procedures must be updated together when behaviour changes. Typed definitions and JSON Schema make interfaces precise enough to check across modules. Defect records must include reproduction steps, severity, expected and observed behaviour, evidence and an owner; closure requires successful retesting by someone other than the author. Handover instructions must cover configuration, schema changes, monitoring, rollback and the local demonstration fallback. Build and test identifiers link stored evidence to the relevant release. Controlled versions and recorded design decisions help another team member investigate or change the system without relying on informal explanations.

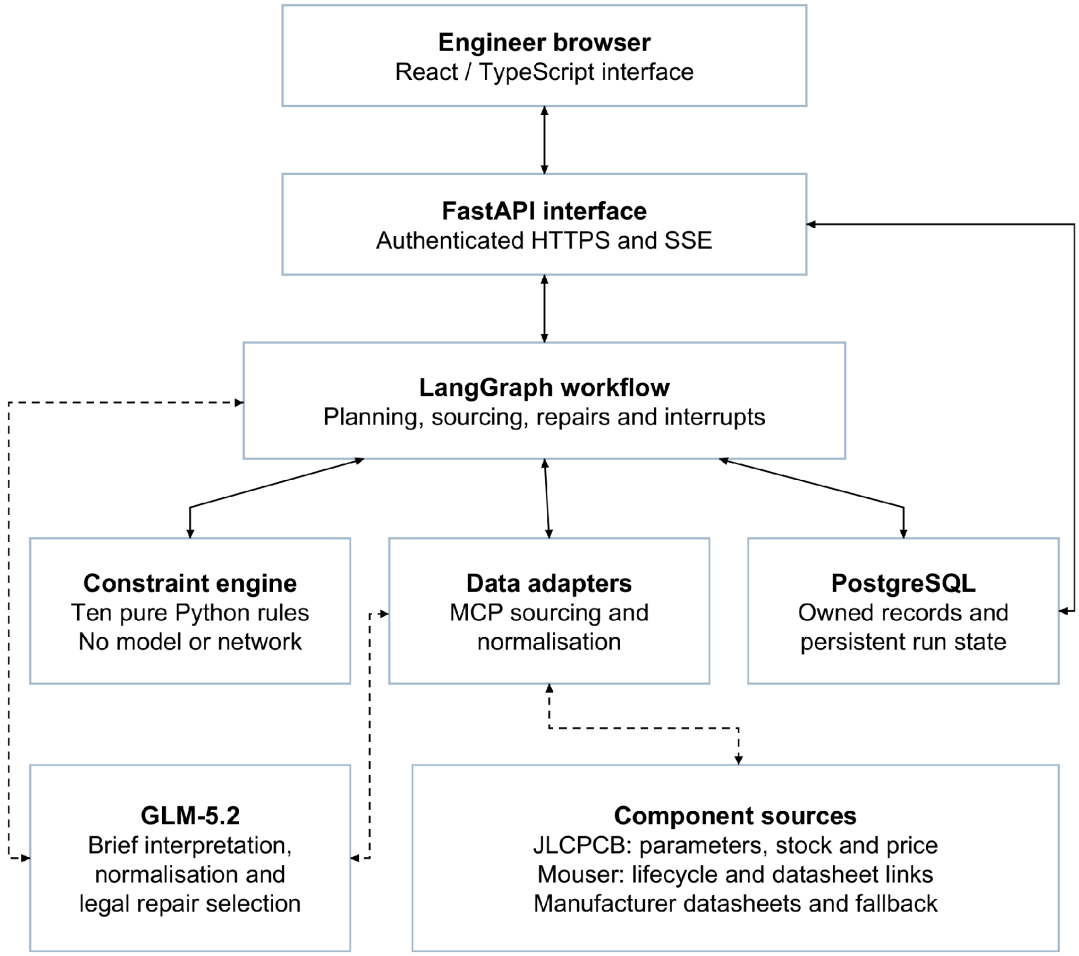
- **Modularity**

FUZE separates its constraint engine, planning, sourcingandnormalisation,
workflow orchestration, persistence and API responsibilities. External component services sit behind MCP adapters, so supplier changes can be addressed without altering electrical rules. The frontend separates account and project screens, streaming, graph, BOM and conflict views. Analytics reads stored findings and events independently of validation and external services. Maintenance should preserve these boundaries through isolated rule tests, recorded supplier responses, interface checks and end-to-end tests.

2. Architectural Design Patterns

FUZEusesabrowser-and-serverarchitecturewithaseparatedomainengineanda stateful workflow. The browser presents the engineer's work, FastAPI handles authenticated requests, and LangGraph coordinates design execution. External services are reached through adapters. PostgreSQL retains the records needed for saved views and recovery.

The following design shows the main maintenance boundaries. The constraint engine receives explicit board data and owns validation decisions. Project browsing, saved views and analytics can read stored records through the application interface without executing a design workflow.



FUZE architecture and maintenance boundaries. Dashed arrows show external-service exchanges.

Presentation and application services. The frontend uses React 19, TypeScript, Vite and Tailwind CSS. It separates account and project screens from the validation trace, component graph, conflict drawer, BOM, memory and analytics views. FastAPI runs on Python 3.11 or later and exposes JSON over HTTPS and Server-Sent Events (SSE). Input validation and resource ownership belong at this boundary, allowing presentation changes to be checked without altering electrical rules.

Deterministic constraint engine. The independent Python engine applies the complete rule set after every placement and applied repair. Its pure functions have no network, model or system-clock dependency, so the same supplied board data can be evaluated repeatedly. This makes an incorrect verdict reproducible and allows a rule correction to be tested separately from catalogue availability or model behaviour.

Constraint rules and their maintenance requirements

Rule	Behaviour to preserve
R1 Voltage overlap	Check the assigned rail against the part's published supply-voltage range.
R2 Current budget	Check total peak current against source capacity with a 15% margin. Missing current must not be treated as zero.
R3 Thermal dissipation	Estimate regulator losses and temperature rise using supported thermal data, ambient temperature and applicable efficiency. Missing required inputs remain unchecked.
R4 Interface role match	Check protocol family, complementary roles and compatible signal levels or a suitable level translator.
R5 Pin budget	Compare required pins with available microcontroller pins after fixed functions. Count a shared bus once.
R6 Availability	Compare stock with required build quantity. Insufficient stock fails; obsolete status or a part not recommended for new designs warns.
R7 Temperature rating	Check that the part's published range covers the application's operating-temperature range.
R8 Footprint	Warn about packages unsuitable for the intended assembly process. This rule is advisory and never fails a board.
R9 Energy budget	Compare battery capacity and continuous current draw with the required runtime when those inputs are available.
R10 Rail coverage	Mark an unassigned part as unchecked so that missing power connections cannot appear valid.

Every verdict is pass, fail, warning or unchecked and retains the deciding field, value and source information. Missing required data remains unchecked; maintenance must

not introduce numerical defaults that create a false pass. If the engineer accepts a conflict, the check still runs and the accepted condition is recorded as a warning. Separate advisory checks flag operation above 80% of a rated limit or a calculated temperature rise above 60 degrees Celsius; they add no pass/fail rules.

External adapters and normalisation. The Model Context Protocol (MCP) tool layer separates supplier responses from the typed specifications used by the engine. JLCPCB provides parameters, stock and price; Mouser provides lifecycle information and datasheet links. Manufacturer datasheets and web-search fallback support missing evidence. GLM-5.2 interprets briefs, normalises catalogue text and selects permitted repairs; validation verdicts remain the engine's responsibility.

Physical-property interpretations are cached by manufacturer part number and a fingerprint of the normalisation prompt. Changing the prompt invalidates the affected interpretation. Stock, lifecycle information and datasheet links are retrieved live for each run, then retained with their observation time. This preserves both current sourcing and reproducible historical results. An unresolved part or absent field must remain visible.

Datasheet uploads enter the same evidence path. The PDF, part number and package identify the requested component, and a usable extracted thermal value is stored with its source line. If extraction fails, the existing gap remains unchecked. Changes to parsing or accepted file formats therefore require input-validation and provenance checks.

Workflow and bounded repair. LangGraph coordinates planning, sourcing, validation, repairs and escalation, with persistent checkpoints for recovery. The engine determines which participants may change and supplies the legal actions. These include swapping a part, changing topology, adding a part, changing a rail, relaxing a requirement and escalating to the engineer. Explicitly requested slots are limited to swapping, changing the rail or escalation. The conditions for requirement relaxation still need to be finalised.

A model choice must belong to the legal set before it is applied. Illegal or timed-out choices use the permitted deterministic fallback. Every applied action triggers full-board revalidation. A slot allows at most three repairs, and a run may add at most three extra slots. These counters survive checkpoints, preventing recovery from restarting an exhausted repair loop.

An unresolved conflict can pause the workflow for an engineer's answer. Resuming that interrupt is separate from continuing an abandoned or errored run from its last completed checkpoint. Keeping these states distinct prevents an answer, repeated request or reconnect from starting the wrong operation.

Ordered events and recoverable views. SSE carries sourcing progress, selections, verdicts, repairs, questions and completion status to the browser. Concurrent work feeds

one ordered queue per run, with increasing sequence numbers and a persisted high-water mark. The client remembers its last rendered sequence and discards duplicates on reconnect. This lets the team investigate a missing update against the stored event history.

The trace, graph, conflict drawer and BOM display that run state. Loading, empty data, failed retrieval and a stalled connection require distinct messages. Saved-board viewing and browsing completed traces are read-only. Final CSV export reflects the post-repair BOM. The output contract still needs per-row verdict and unchecked-rule fields, which require corresponding export tests.

Persistence and access control. PostgreSQL stores accounts, projects, runs, boards and workflow state, including the events, repair counters and evidence needed for recovery. Application requests must check authentication and ownership before reading or changing protected data. The security requirements specify Argon2id password hashing, secure session cookies and credentials kept outside version control. Failed operations must not appear as successfully saved changes.

Database schema changes use idempotent statements at application startup. Each change needs restart and compatibility checks, together with a documented reversal procedure. A code rollback must also account for the stored schema and data. Project deletion and other unresolved retention behaviour require an explicit contract before they are changed.

Memory and analytics. Memory browsing exposes remembered parts, the boards in which they appear and recorded findings without changing them. Separately, the repair workflow can retrieve up to three previously successful repairs for a matching conflict signature. These examples provide context for selection; the legal-action check and engine revalidation still apply.

Analytics computes four views from the authenticated engineer's completed runs: rule-outcome distribution, repair effectiveness by action, recurring conflict signatures and supply exposure. It reads stored findings and events without calling suppliers, the model or the engine, and persists no derived score. Abandoned and errored runs are excluded. Each rate must retain its numerator, denominator and unchecked count, with an explicit empty state when no observations exist. The proposed 20% unchecked threshold for withholding a rate remains subject to confirmation.

Deployment and operational maintenance. The deployment design specifies HTTPS hosting for a static browser application, one SSE worker and managed PostgreSQL. Render is the selected platform, with a local demonstration fallback. Runbooks must cover startup, secrets, schema changes, health checks and recovery. Multi-server scaling needs separate coordination of run ownership and events and remains outside scope.

3. Software Configuration Management Tools

Configuration management connects an application release with its source, dependencies, prompts, schema, tests and operating instructions. FUZE uses GitLab for code and configuration, Trello for work tracking, and Google Drive for shared documents and supporting evidence.

3.1. GitLab

The campus GitLab repository is the shared code baseline, accessed through the NTU network or VPN. Commit history, reviewed changes and release tags help maintainers identify and reverse changes. Each implementation should be linked to its requirement or defect and verification results.

Code, contracts, normalisation prompts, schema statements and test fixtures must be versioned together. Pinned dependencies support repeatable builds; credentials must be supplied through protected environment settings. Automated coverage enforcement is planned and must be verified before it is relied upon as a merge gate.

3.2. Trello

Trello provides a shared view of tasks, owners, progress and milestone actions, alongside GitLab issues and the team backlogs. Defects need reproduction steps, expected and observed behaviour, severity, evidence and an owner. They move through triage, implementation and retesting before closure; a failed retest reopens the issue.

Regular team reviews keep unresolved work and risks visible. Mohika coordinates changes that affect scope or acceptance, records their effect on effort and schedule, and obtains sponsor approval before the baseline is updated. Completing a task card must be supported by the relevant review and test result.

3.3. Google Drive and Controlled Documentation

Google Drive holds shared proposals, requirements, use-case models, design documents, quality and risk plans, meeting minutes and supporting evidence. Each document revision must identify the application release to which it applies.

Milestone archives must include the release reference, test results, known issues and handover guidance. Release evidence requires a second copy, verified access before handover and retention for one year afterwards.